Ceph Performance Evaluation Report

Issei Hasegawa

2026-05-07

Table of contents

1 Overview

This study evaluates the performance of Ceph, a distributed storage system, by utilizing a block I/O trace. Ceph is designed to unify storage devices across various nodes, providing both durability and scalability. In particular, the RADOS Block Device (RBD) enables the creation of virtual block devices on top of Ceph, which can be accessed for reading and writing just like standard disks.

The purpose of this study is to replay a vSCSI block I/O trace named cloudPhysicsIO.vscsi on a Ceph cluster set up on CloudLab, while measuring and analyzing its performance throughout the replay process. Rather than relying on artificial benchmarks such as basic random reads or writes, this study employs a trace that more accurately reflects real-world storage access patterns. This approach allows for the observation of Ceph's behavior under a more authentic workload.

Regarding the performance evaluation, it primarily collects metrics such as throughput, IOPS, latency, CPU usage, memory usage, network usage, and disk usage.Using these metrics, we assess the performance of the Ceph cluster, identify any bottlenecks, and determine whether the observed results are reasonable given the workload and Ceph configuration.

This paper first describes the experimental environment and the configuration of the Ceph cluster, followed by an explanation of the characteristics of the vSCSI traces used and the measurement methods employed. Finally, based on the results obtained, we analyze the performance characteristics of Ceph and discuss the limitations of this experiment and areas for future improvement.

2 Experimental Environment

In this study, three nodes were used on CloudLab and a Ceph cluster was set up with them. Each node has same hardware specifications, which are used to evaluate the perfomance of Ceph as distributed storage system. In this study, the storage were set up across multiple nodes, and an actual block I/O trace was replayed on the Ceph RBD. Because of this, the porpose of this expeirment is not to observe the simple simulation, but to evaluate the I/O performance of Ceph under a real-world distributed storage environment.

2.1 Hardware Environment

This study used three nodes on CloudLab's FASTA25-AE project. Used nodes, 'node0', 'node1', and 'node2', have the same hardware specifications. Each node is equipped with an Intel Xeon E5-2630 v3 CPU, 64 GB of DDR4 memory, an SSD for the operating system, and HDDs used as Ceph OSDs. Specifically, each node has two HDDs, for a total of six HDDs used as Ceph OSDs. Additionally, a 10 GbE network was used for communication between nodes.

Component	Specification
Platform	CloudLab (FAST25-AE project)
Number of nodes	3 (`node0`, `node1`, `node2`)
CPU	Intel Xeon E5-2630 v3 @ 2.40 GHz, 8 cores
Memory	64 GB DDR4 2133 MHz ECC Registered
OS Disk	Intel SSDSC2BX20 200 GB SSD
OSD Disk	Seagate ST1000NM0033 HDD × 2 per node, 931 GB each
Network	Intel X710 10 GbE SFP+

With this configuration, Ceph stores data using HDDs distributed across multiple nodes. Consequently, the performance results of this experiment are influenced not only by the performance of individual disks but also by HDD access speeds, the inter-node network, and Ceph's replication processes.

2.2 Software Environment

This study used Ubuntu 22.04.2 LTS on each node, and installed Ceph 17.2.9 Quincy to set up the cluster. For Ceph storage, RBD was used and vSCSI was replayed for created RBD. Regarding the trace replay, an originally created Python script was used, which reads block I/O called 'cloudPhysicsIO.vscsi' and issues actual read/write requests to '/dev/rbd0'. In addition, in order to analyze the cash performance, 'cachesim' from libCacheSim was utilized. The software environment is summarized in the following table:

Component	Version / Detail
Operating System	Ubuntu 22.04.2 LTS, Linux 5.15.0-168-generic
Ceph	17.2.9 Quincy
Trace replay tool	Custom Python script using `O_SYNC` direct I/O to `/dev/rbd0`
Cache simulation tool	libCacheSim `cachesim`
Trace file	`cloudPhysicsIO.vscsi`

This software environment enables a comparison between the I/O performance on an actual Ceph cluster and the behavior predicted by cache simulations.

3 Ceph Configuration

In this experiment, a Ceph cluster was set up using three nodes. It manages storage devices on multiple nodes, distibuted storage system that distributes, replicates, and stores data. This section describes the configuration of the Ceph cluster and the setting of storage used in this experiment.

3.1 Cluster Topology

The Ceph cluster in this experiment consists of three nodes, which are 'node0', 'node1', and 'node2'. Each node hosted a MON daemon, forming a quorum of three monitors. Additionally, MGR daemon was configured with 'node0' as the active manager and 'node1' as the standby manager.

Six OSDs were set up in total. Each node had two HDD and each HDD was used as a Ceph OSD. Specifically, 'osd0' and 'osd.2' were palced on 'node0', 'osd.1' and 'osd.3' were placed on 'node1', and 'osd.4' and 'osd.5' were placed on 'node2'. BluesStore was used as the OSD backend. The Ceph cluster configuration is summarized below.

Component	Configuration
MON daemons	`node0`, `node1`, `node2`
MGR daemons	`node0` active, `node1` standby
OSD count	6 total
OSD placement	`node0`: `osd.0`, `osd.2`; `node1`: `osd.1`, `osd.3`; `node2`: `osd.4`, `osd.5`
OSD backend	BlueStore
Replication factor	3
Pool	`rbd` with 32 PGs
RBD image	`trace-disk`, 35 GB
Mapped device	`/dev/rbd0` on `node0`
BlueStore cache size	1 GB per OSD
Total raw capacity	5.5 TiB
Cluster health	`HEALTH_OK`

In this configuration, all of three nodes are related to both the cluster management and data storage.

3.2 Storage Configuration

This experiment utilized Ceph RBD to establish a block device. A 35 GB RBD image, designated as `trace-disk`, was created and mapped on `node0` as `/dev/rbd0`. The trace replay workload was performed on this mapped RBD device.

The replication factor was configured to 3. This indicates that each data segment is replicated across three OSDs. Such a configuration enhances durability, as the system can withstand certain OSD or node failures without data loss. However, it may also lead to increased write latency since each write operation must be replicated to multiple OSDs before receiving acknowledgment.

Each OSD employed BlueStore as its storage backend. BlueStore serves as the standard Ceph backend for the management of data blocks, metadata, and RocksDB-related information. In this experiment, the BlueStore cache size was allocated at 1 GB per OSD. Nevertheless, this cache is not exclusively reserved for user data; it is also utilized for metadata and RocksDB-related data. Consequently, the actual cache capacity available for user data may be less than the total 1 GB.

In summary, this storage configuration integrates three nodes, six OSDs, a replication factor of 3, and RBD to replay the trace within a realistic distributed storage environment.

4 Workload Analysis

In this experiment, we used a vSCSI block I/O trace named `cloudPhysicsIO.vscsi` as the workload. This trace records storage access patterns observed in actual cloud environments. As a result, it can reproduce I/O behavior in a Ceph cluster that is closer to real-world conditions than artificial benchmarks such as simple random read/write operations.

In this chapter, we first explain the format of the trace file, and then summarize the workload characteristics contained in the trace, such as the ratio of reads to writes, the number of requests, I/O size, and address space.

4.1 Trace Format

The trace file used, `cloudPhysicsIO.vscsi`, is a binary vSCSI trace. In this file, each I/O request is stored as a 32-byte record. Each record contains information such as the request number, I/O size, SCSI command, logical block number, and timestamp.

The record format of the trace is as follows.

Field	Size	Description
`sn`	4 bytes	Sequence number
`len`	4 bytes	I/O size in bytes
`nSG`	4 bytes	Scatter-gather count
`cmd`	2 bytes	SCSI command code
`ver`	2 bytes	Version field
`lbn`	8 bytes	Logical Block Number
`ts`	8 bytes	Timestamp in microseconds

In this context, the `cmd` field was used to determine whether each request was a read or a write. Additionally, `lbn` indicates the logical block number of the target, and when actually issuing I/O to the Ceph RBD, this value was used as an offset in 512-byte units.

4.2 Workload Characteristics

This trace contains a total of 113,872 I/O requests. Of these, 46,974 were read requests and 66,898 were write requests, indicating that the proportion of writes is higher than that of reads. Therefore, this workload is write-dominant.

The main characteristics of the workload are as follows.

Metric	Value
Total requests	113,872
Read requests	46,974 (41.3%)
Write requests	66,898 (58.7%)
Total read data	1,714 MB
Total write data	2,297 MB
Trace duration	7,200 seconds
Average request rate	15.82 requests/sec
Most frequent I/O size	65,536 bytes
I/O size range	512 bytes to 69,632 bytes
Address space required	Approximately 31.3 GB

In this workload, the most frequently occurring I/O size was 65,536 bytes, or 64 KB. The I/O sizes ranged from 512 bytes to 69,632 bytes. The total duration of the trace was 7,200 seconds, or 2 hours, and the average request rate was approximately 15.82 requests per second.

Furthermore, this workload does not generate requests at regular intervals but exhibits burst-like behavior. While the number of requests is low during most time periods, there are certain periods where a large number of requests are concentrated within a short time frame. This behavior is considered to closely resemble I/O patterns in actual cloud storage environments.

Based on these characteristics, this trace represents a workload with a high write ratio, a skewed access pattern, and bursty behavior. By replaying such a workload on Ceph, we can evaluate how a distributed storage system behaves under realistic I/O patterns.

5 Measurement Method

This chapter describes the methods used to measure the performance of a Ceph cluster. In this experiment, we not only replayed traces against Ceph but also performed cache simulations using libCacheSim. This allowed us to compare the theoretically predicted cache behavior with the actual cache behavior observed in the Ceph BlueStore backend.

The measurements were conducted in three main steps. First, we used libCacheSim to simulate the cache miss rate for the traces using various cache algorithms and cache sizes. Next, we replayed the same traces as actual block I/O operations against a Ceph RBD device. Finally, we collected performance counters from Ceph BlueStore to analyze how many reads actually occurred on the BlueStore side and how many of those resulted in cache hits or cache misses.

5.1 Cache Simulation with libCacheSim

First, we used the `cachesim` tool from libCacheSim to simulate cache behavior for the `cloudPhysicsIO.vscsi` trace. The purpose of this simulation was to determine the likelihood of cache hits for this workload in an ideal cache environment.

In the simulation, we used four types of cache replacement algorithms: LRU, LFU, FIFO, and ARC. We also tested cache sizes of 10 MB, 50 MB, 100 MB, 500 MB, 1 GB, and 2 GB. This allowed us to observe how the miss rate changes with increasing cache size and to identify differences between the various algorithms.

This simulation assumes an ideal scenario where the cache is dedicated solely to data blocks and is not affected by other metadata or background processes. Therefore, the results from libCacheSim were used as a theoretical benchmark for comparison rather than as a direct representation of the actual cache performance of Ceph BlueStore.

5.2 Real I/O Replay on Ceph RBD

Next, we replayed the vSCSI trace as actual block I/O on a Ceph RBD device. To do this, we used a Python script that reads the trace file and interprets each record as a read or write request. The I/O was directed to the Ceph RBD device `/dev/rbd0`, which is mapped on `node0`.

For each request, we calculated the access offset based on the Logical Block Number (lbn) in the trace. Specifically, we multiplied the lbn by 512 bytes to determine the byte-level offset on

the RBD device. We also used the `len` field in the trace for the I/O size. This allowed us to issue actual I/O operations that closely matched the access locations and sizes recorded in the trace.

For write requests, `O_SYNC` was used. This ensures that writes are not merely written to the OS page cache, but are processed by Ceph and completed only after being committed to all three OSD replicas. This configuration is essential for measuring Ceph's actual write latency.

Additionally, the Linux page cache was flushed before starting the replay. This minimized the influence of the page cache at the start of the experiment. Requests were issued in a single-threaded manner, following the order of the traces. This was done to preserve the I/O order within the traces and to first observe the basic behavior of a single client.

5.3 BlueStore Statistics Collection

Finally, we used Ceph BlueStore's performance counters to measure the actual read and caching behavior occurring in the BlueStore backend. We used the `ceph tell osd.* perf dump` command to retrieve the performance counters for each OSD immediately before and after trace replay.

By calculating the difference between the counters before and after replay, we extracted the I/O activity generated by the trace replay. In particular, this experiment focused on counters related to BlueStore reads.

The main counters used are as follows.

Counter	Description
`read_random_count`	Total number of random reads received by BlueStore
`read_random_buffer_count`	Number of reads served from the BlueStore memory cache
`read_random_disk_count`	Number of reads that required disk access

Since `read_random_buffer_count` represents reads processed by BlueStore's memory cache, we treated it as a cache hit. On the other hand, since `read_random_disk_count` represents reads that actually required disk access, we treated it as a cache miss.

This approach allowed us to compare the cache miss rate predicted by libCacheSim with the cache miss rate observed in the actual BlueStore system.